

ChefVision AI

Project Workbook

By

Girith Choudhary

Richard Pham

Shivansh Chhabra

March 4, 2026

Advisor: Dr. Wencen Wu

Chapter 1. Literature Search, State of the Art

Literature Search

ChefVision AI sits at the intersection of:

1. **Document understanding** (invoice/receipt extraction),
2. **Entity normalization** (ingredient name standardization / record linkage), and
3. **Forecasting + planning** (multi-horizon demand prediction + event-based ingredient planning).

Key research themes we are leveraging:

- **Visual Document Understanding (VDU) and Receipt Parsing**
 - Transformer-based document models reduce manual feature engineering and can learn layout + text jointly.
 - Receipt datasets and competitions provide benchmarks for line-item extraction and key field extraction.
- **Entity Matching / Normalization**
 - “Same entity, different name” is a core data management problem; deep learning + embeddings improve matching robustness over rules alone.
 - ChefVision’s normalization layer combines similarity scoring, rules (units/vendor patterns).
- **Forecasting (Short-Term + Multi-Horizon)**
 - Practical forecasting systems emphasize robustness, seasonality, and evaluation workflows.
 - Deep multi-horizon models (attention-based) can incorporate static covariates and known future inputs.
 - Gradient boosting remains strong for tabular forecasting features and cold-start regimes.

How the literature maps to our 3-model plan:

- **Model 1 (Transformer):** Document extraction from invoice/receipt images
- **Model 2 (ML):** Recipe–ingredient modeling for usage/costing knowledge (plus curated recipes)
- **Model 3 (ML + planning logic):** Forecasting + event-based ingredient requirements.

State-of-the-Art Summary

Modern “Document AI” has moved beyond basic OCR into **structured extraction** (tables, key-value pairs, line items) and **domain-specific parsers**. Leading-edge approaches increasingly combine:

- **Multimodal Transformers** (text + layout + vision) for robust extraction across formats
- **OCR-free or OCR-light pipelines** to reduce error propagation and simplify engineering
- **Human-in-the-loop review** for low-confidence fields to keep accuracy high at reasonable cost
- **Entity resolution + embeddings** to normalize noisy vendor naming and unify catalogs
- **Multi-horizon forecasting** to incorporate seasonality, local signals, and event schedules

In restaurant operations, existing inventory platforms primarily focus on inventory counts, ordering workflows, and recipe costing. ChefVision AI differentiates by making photo-first ingestion + ingredient normalization + event-based planning the center of the workflow:

- Start with photos/CSVs in minutes (low onboarding friction)
- Normalize supplier naming differences into a canonical ingredient catalog
- Convert events and recipes into ingredient requirement plans (week-by-week)
- Generate actionable reorder recommendations and shortage alerts

References

1. Geewook Kim, Teakgyu Hong, Moonbin Yim, JeongYeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, Seunghyun Park. **“OCR-free Document Understanding Transformer (Donut).”** *European Conference on Computer Vision (ECCV)*, 2022. (Also available as arXiv:2111.15664).
Description: A model that reads invoice/document images and extracts structured data without needing a separate OCR step. It converts raw images directly into structured JSON output.
2. Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, Furu Wei. **“LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking.”** arXiv:2204.08387, 2022. (arXiv)
Description: A pre-trained model that understands documents by learning from text, layout positions, and images together. Useful for extracting key fields from invoices where position on the page matters.
3. Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, Vijay Raghavendra. **“Deep Learning for Entity Matching: A Design Space Exploration.”** *SIGMOD 2018*, pp. 19–34. DOI: 10.1145/3183713.3196926. (DBLP)
Description: A study on using deep learning to decide whether two different text descriptions refer to the same real-world item. Relevant for matching supplier ingredient names to a standard catalog.
4. Bryan Lim, Sercan O. Arik, Nicolas Loeff, Tomas Pfister. **“Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting.”** arXiv:1912.09363, 2019. (arXiv)
Description: A forecasting model that predicts future values across multiple time steps using historical data and known future inputs. Can be used to forecast ingredient demand based on upcoming events and past usage patterns.

Chapter 2. Project Justification

ChefVision AI solves a common, high-impact operational problem: **inventory decisions are only as good as the data behind them**, and that data is often unstructured (paper invoices, inconsistent item naming, scattered notes).

Why this matters:

- **Financial impact:** Mis-ordering drives waste, margin loss, and staff time spent reconciling invoices manually.
- **Operational reliability:** Stockouts create lost sales and degraded customer experience, especially during demand spikes.
- **Complexity increases over time:** More suppliers, changing pricing, and varying naming conventions create long-term data fragmentation.

Why ChefVision AI is the right approach:

- **Photo-first ingestion** matches the real workflow (owners already receive paper invoices and take photos).
- **Normalization is the multiplier:** Once “tomato / tomotoh / roma tomato” becomes one canonical ingredient, every downstream function improves (inventory accuracy, costing, forecasting).
- **Event-based planning is actionable:** Restaurants often plan around catering/events; converting events + recipes into ingredient requirements creates immediate value even before forecasting matures.
- **Scales from MVP to product:** Start with ingestion + normalization + Excel outputs; expand to integrations and predictive analytics as data accumulates.

Success criteria:

- Reduce manual invoice processing time
- Improve ordering accuracy (fewer stockouts and less overstock)
- Provide clear, explainable reorder suggestions tied to inventory + planned events

Chapter 3: Identify Baseline Approaches

Baselines for Document Extraction (Invoice → Line Items)

- **Baseline A (Classic OCR pipeline):** OCR (e.g., Tesseract) + rule-based parsing (regex/heuristics for totals, dates, and table-like lines).
 - Strength: simple, fast to implement
 - Weakness: brittle across varying layouts and photo quality
- **Baseline B (Layout-aware Transformer):** LayoutLMv3 fine-tuning for key-value/line-item extraction.
- **Proposed (Primary):** Donut-style end-to-end extraction with confidence scoring + user review for low-confidence fields.

Baselines for Ingredient Normalization (Entity Matching)

- **Baseline A (String-only):** normalized edit distance + token overlap + Jaro-Winkler; merge above threshold.
- **Baseline B (Embedding similarity):** sentence/word embeddings + cosine similarity + simple thresholding.
- **Proposed:** hybrid match = embeddings + string metrics + unit/vendor rules + confidence gating; guided by entity matching insights.

Baselines for Forecasting / Reorder Quantity

- **Baseline A (Naive):** moving average / seasonal naive (yesterday/last-week same-day).
- **Baseline B (Classical):** Prophet-like trend/seasonality model (if adopted later).

Chapter 4. Dependencies and Deliverables

Dependencies

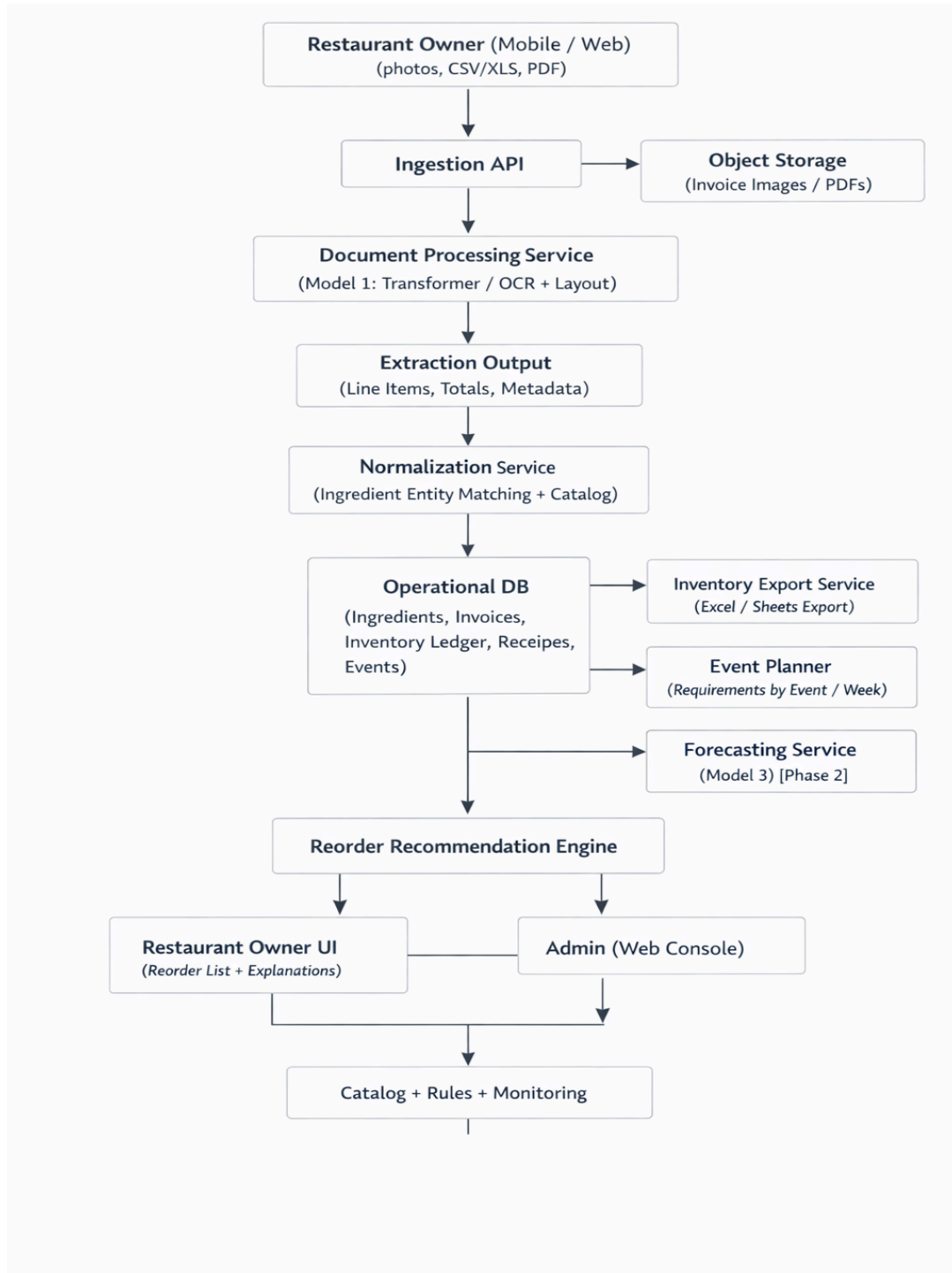
- Access to **representative invoices/receipts** for development/testing (real or sanitized).
- A **baseline ingredient catalog** (seed data) to support normalization early.
- Compute environment for model training/inference (GPU helpful for doc models).
- Storage for document images and extracted structured data.
- Recipe–ingredient dataset availability (public dataset + optional curated restaurant recipes).

Deliverables

- **Restaurant Owner App (User UI):** upload invoices, review extraction, inventory view/export, event planner, reorder list
- **Admin Console (Admin UI):** canonical ingredient catalog management, synonym rules, model/config controls
- **Backend API + Database:** authentication, data storage, business logic
- **ML Services**
 - Model 1: document extraction service
 - Normalization service: matching + confidence scoring
 - Model 3: forecasting (when enabled)
- Excel/Spreadsheet Export Template for inventory and planning outputs
- Demo Dataset + End-to-End Demo showing photo → inventory → event plan → reorder list.

Chapter 5. Project Architecture

Diagram



Description

ChefVision AI uses a client-server architecture. Users upload invoices as photos (primary ingestion) and optionally upload files (CSV/Excel/PDF). Documents are stored in object storage, then processed by a Document AI service (Model 1) that extracts structured line items. Extracted items are passed into a normalization service that resolves naming variants to a canonical ingredient catalog using embeddings + rules + confidence scoring. Cleaned inventory updates are stored in the operational database and can be exported to an Excel-compatible sheet.

On top of this cleaned data layer, the application supports event planning: users specify events, recipes, and serving counts; the system calculates ingredient requirements and checks inventory sufficiency. If shortages exist, the reorder engine generates a smart reorder list with explainable quantities (tied to event plans and historical usage). In later phases, a forecasting service (Model 3) provides short-term demand estimates to refine reorder quantities beyond events.

Chapter 6: Evaluation Methodology

Testbed Setup

- **Data sources**
 - Invoice/receipt images from multiple suppliers
 - Optional public document datasets for pretraining sanity checks (if used)
 - Recipe + ingredient lists (public + curated)
- **Environment**
 - ML inference service + backend + DB
 - Controlled evaluation scripts for repeatability (fixed seeds, logged configs)

Evaluation Tasks + Metrics

1) Document Extraction Accuracy

Goal: correctly extract structured line items.

- **Field-level metrics:** precision/recall/F1 for vendor/date/totals
- **Line-item metrics:** exact match rate for (name, qty, unit, price) with tolerances (e.g., numeric tolerance for price)
- **End-to-end utility metric:** % of invoices requiring manual edits (human-in-loop rate)

Baselines:

- OCR + heuristics baseline
- LayoutLMv3-based baseline
- Donut-based model

2) Ingredient Normalization Accuracy

Goal: merge “same ingredient” variants correctly.

- **Pairwise matching precision/recall/F1** on a labeled alias set
- **Cluster quality:** number of duplicates remaining vs false merges
- **Human correction rate:** how often users override a suggested match

Baselines:

- string-only matching
- embedding-only matching

3) Planning/Reorder Effectiveness

Goal: produce correct reorder quantities for event needs and (later) forecast needs.

- **Shortage detection accuracy:** did the system correctly flag missing ingredients for an event plan?
- **Reorder list correctness:** error in required quantities (absolute/percentage)
- **Weekly planning accuracy:** variance between planned usage vs actual usage (when historical data exists).

Chapter 7: System Design — Solution Overview

End-to-End Workflow

1. **Invoice Photo Upload**
 - owner takes photos of delivery invoices and uploads
2. **Model 1: Document Extraction (Transformer)**
 - extract vendor/date/totals + line items
 - output confidence per field/line item
3. **Ingredient Normalization**
 - detect duplicates (“tomato” vs “tomotoh”)
 - map to canonical ingredient ID
 - low-confidence → user confirmation flow
4. **Inventory Ledger + Excel Export**
 - write cleaned, standardized inventory updates
 - export Excel-compatible sheet
5. **Event Planning**
 - owner enters upcoming events and recipes + serving count
 - compute ingredient requirements event-by-event and optionally week-by-week
6. **Reorder Recommendations**
 - compare requirements vs current inventory

- output reorder list + explanation (why each quantity is needed)

Chapter 8. Implementation Plan and Progress

Implementation Plan

Phase 0: Setup

- Repo + branching strategy, environment setup
- DB schema draft, API skeleton, basic auth/roles

Phase 1: Ingestion + Extraction MVP

- Mobile/web upload flow
- Baseline extraction pipeline (Model 1 integration)
- Storage + job queue + retry logic
- UI for reviewing extracted line items

Phase 2: Normalization + Inventory Ledger

- Canonical ingredient catalog + alias table
- Matching logic
- Inventory ledger updates + Excel export

Phase 3: Recipe + Event Planning

- Recipe/ingredient tables + serving scaling
- Event planner UI
- Requirements computation and reorder list output

Phase 4: Forecasting

- Feature engineering from ledger + calendar + events
- Baseline models (boosting) then upgrade as data grows
- Backtesting + monitoring

Current Progress (as of March 4, 2026)

- Project scope defined: photo-first invoice ingestion → normalization → inventory sheet → event planning → reorder list.
- Three-model roadmap established (Document AI, recipe/ingredient model, forecasting + planning engine).

Chapter 9. Project Schedule

Sprint Plan

Sprint 0 (Mar 4 – Mar 10): Foundations

- Define MVP scope, user stories, acceptance criteria
- Set up repo, CI, dev environments, initial DB schema

Sprint 1 (Mar 11 – Mar 24): Ingestion + Job Pipeline

- Photo upload + storage
- Processing job queue + status tracking
- Stub UI for invoice list + upload

Sprint 2 (Mar 25 – Apr 7): Document Extraction (Model 1)

- Integrate extraction model/service
- Store extracted line items in DB
- Review UI (edit line items)

Sprint 3 (Apr 8 – Apr 21): Normalization + Catalog

- Canonical ingredient catalog + alias management
- Matching + confidence scoring
- “Confirm match” UI and audit trail

Sprint 4 (Apr 22 – May 5): Inventory Ledger + Excel Export

- Ledger updates from normalized line items
- Excel export generator + downloadable report
- Basic analytics (top spend, frequent ingredients)

Sprint 5 (May 6 – May 19): Event Planner + Reorder List

- Events + recipes + serving count
- Ingredient requirement calculator
- Reorder list generation (event-by-event + weekly summary)

Sprint 6 (May 20 – Jun 2): Hardening + Demo

- QA pass, performance tuning, bug fixes
- Final demo script and documentation

Team Contributions

- **Richard Pham:** Document extraction (Model 1), dataset prep, evaluation metrics

- **Girith Choudhary:** Ingredient normalization service, canonical catalog design, matching evaluation
- **Shivansh Chhabra:** Full-stack integration (UI/API), deployment pipeline, Excel export, testing harness